Dat Bike External Grab API Guide

Document Version: 1.4

Last Updated: August 6, 2026

This document describes the CAN bus API for integrating external devices with Dat Bike electric motorcycles. This API allows external devices to receive bike telemetry data and participate in the secure deservice/reservice protocol, including an explicit deservice/reservice command.

Table of Contents

1. Hardware Setup
2. CAN Bus Configuration
3. CAN Frame Format
4. Message Encoding (Protocol Buffers)
5. API Messages
6. Deservice/Reservice Command
7. Deservice/Reservice Protocol
8. Test Board Usage

Hardware Setup

Physical Connection

- **CAN Bus**: Standard CAN 2.0B with extended frames (29-bit identifier)
- **Baud Rate**: 500 kbps
- **Termination**: The bike's CAN bus is already properly terminated with 120Ω resistors. **Do NOT add additional termination resistors** on your device, as this would lower the bus impedance and cause communication errors.

A test board with a pre-wired CAN connector is provided for development and testing. Unlike the bike (which has both 120Ω termination resistors), the test board only has **one 120Ω resistor**. You must add a 120Ω termination resistor on your device when connecting to the test board.

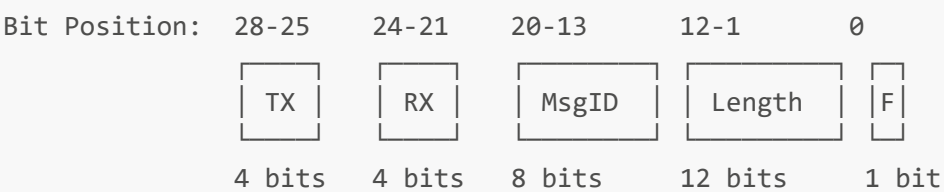
CAN Bus Configuration

Parameter	Value
Protocol	CAN 2.0B Extended
Identifier	29-bit extended
Baud Rate	500 kbps
Frame Type	Data frames only

CAN Frame Format

29-bit Extended Identifier Structure

The 29-bit CAN identifier is structured as follows:



Field	Bits	Description
Sender ID	28-25 (4 bits)	Source board ID
Receiver ID	24-21 (4 bits)	Destination board ID
Message ID	20-13 (8 bits)	Message type identifier
Length	12-1 (12 bits)	Total payload length in bytes
IsFirst	0 (1 bit)	1 = first frame of message, 0 = continuation

Board IDs

Your device should:

- **Listen for messages from**: Board ID **1** (VCU - Vehicle Control Unit)
- **Send messages as**: Board ID **6** (External Device)

Board	ID
VCU (Vehicle Control Unit)	1
External Device	6

Message IDs

Message	ID	Direction	Description
GrabInfoMsg	21	VCU → Device	Bike telemetry (every 5 seconds)
GrabKeyRequestMsg	22	VCU → Device	Request key for authentication
GrabKeyResponseMsg	23	Device → VCU	Key response
GrabChallengeMsg	24	VCU → Device	Challenge token for verification
GrabResponseMsg	25	Device → VCU	Hashed token response
GrabDeserviceCommandMsg	27	Device → VCU	Deservice or reservice the bike

Message	ID	Direction	Description
GrabDeserviceResponseMsg	28	VCU → Device	Result of a deservice/reservice command

Multi-Frame Messages

Messages larger than 8 bytes are split across multiple CAN frames:

1. **First frame**: `IsFirst = 1`, `Length = total payload size`
2. **Continuation frames**: `IsFirst = 0`, `Length` decrements by bytes sent

Example for a 20-byte message:

```
Frame 1: IsFirst=1, Length=20, Data[0-7]
Frame 2: IsFirst=0, Length=12, Data[8-15]
Frame 3: IsFirst=0, Length=4,  Data[16-19]
```

CRC16 Checksum (Important)

All messages must include a 2-byte CRC16 checksum appended to the payload.

The CRC is calculated over the protobuf-encoded data and appended in **big-endian** byte order. The `Length` field in the CAN identifier must include these 2 CRC bytes.

CRC Algorithm

The CRC uses the **CRC-16/CCITT (XModem)** polynomial with the following lookup table:

```
static const uint16_t crc16_tab[] = {
    0x0000, 0x1021, 0x2042, 0x3063, 0x4084, 0x50a5, 0x60c6, 0x70e7,
    0x8108, 0x9129, 0xa14a, 0xb16b, 0xc18c, 0xd1ad, 0xe1ce, 0xf1ef,
    0x1231, 0x0210, 0x3273, 0x2252, 0x52b5, 0x4294, 0x72f7, 0x62d6,
    0x9339, 0x8318, 0xb37b, 0xa35a, 0xd3bd, 0xc39c, 0xf3ff, 0xe3de,
    0x2462, 0x3443, 0x0420, 0x1401, 0x64e6, 0x74c7, 0x44a4, 0x5485,
    0xa56a, 0xb54b, 0x8528, 0x9509, 0xe5ee, 0xf5cf, 0xc5ac, 0xd58d,
    0x3653, 0x2672, 0x1611, 0x0630, 0x76d7, 0x66f6, 0x5695, 0x46b4,
    0xb75b, 0xa77a, 0x9719, 0x8738, 0xf7df, 0xe7fe, 0xd79d, 0xc7bc,
    0x48c4, 0x58e5, 0x6886, 0x78a7, 0x0840, 0x1861, 0x2802, 0x3823,
    0xc9cc, 0xd9ed, 0xe98e, 0xf9af, 0x8948, 0x9969, 0xa90a, 0xb92b,
    0x5af5, 0x4ad4, 0x7ab7, 0x6a96, 0x1a71, 0x0a50, 0x3a33, 0x2a12,
    0xdbfd, 0xcbdc, 0xfbbf, 0xeb9e, 0x9b79, 0x8b58, 0xbb3b, 0xab1a,
    0x6ca6, 0x7c87, 0x4ce4, 0x5cc5, 0x2c22, 0x3c03, 0x0c60, 0x1c41,
    0xedae, 0xfd8f, 0xcdec, 0xddcd, 0xad2a, 0xbd0b, 0x8d68, 0x9d49,
    0x7e97, 0x6eb6, 0x5ed5, 0x4ef4, 0x3e13, 0x2e32, 0x1e51, 0x0e70,
    0xff9f, 0xefbe, 0xdfdd, 0xcffc, 0xbf1b, 0xaf3a, 0x9f59, 0x8f78,
    0x9188, 0x81a9, 0xb1ca, 0xa1eb, 0xd10c, 0xc12d, 0xf14e, 0xe16f,
    0x1080, 0x00a1, 0x30c2, 0x20e3, 0x5004, 0x4025, 0x7046, 0x6067,
    0x83b9, 0x9398, 0xa3fb, 0xb3da, 0xc33d, 0xd31c, 0xe37f, 0xf35e,
    0x02b1, 0x1290, 0x22f3, 0x32d2, 0x4235, 0x5214, 0x6277, 0x7256,
    0xb5ea, 0xa5cb, 0x95a8, 0x8589, 0xf56e, 0xe54f, 0xd52c, 0xc50d,
```

```
    0x34e2, 0x24c3, 0x14a0, 0x0481, 0x7466, 0x6447, 0x5424, 0x4405,
    0xa7db, 0xb7fa, 0x8799, 0x97b8, 0xe75f, 0xf77e, 0xc71d, 0xd73c,
    0x26d3, 0x36f2, 0x0691, 0x16b0, 0x6657, 0x7676, 0x4615, 0x5634,
    0xd94c, 0xc96d, 0xf90e, 0xe92f, 0x99c8, 0x89e9, 0xb98a, 0xa9ab,
    0x5844, 0x4865, 0x7806, 0x6827, 0x18c0, 0x08e1, 0x3882, 0x28a3,
    0xcb7d, 0xdb5c, 0xeb3f, 0xfb1e, 0x8bf9, 0x9bd8, 0xabbb, 0xbb9a,
    0x4a75, 0x5a54, 0x6a37, 0x7a16, 0x0af1, 0x1ad0, 0x2ab3, 0x3a92,
    0xfd2e, 0xed0f, 0xdd6c, 0xcd4d, 0xbdaa, 0xad8b, 0x9de8, 0x8dc9,
    0x7c26, 0x6c07, 0x5c64, 0x4c45, 0x3ca2, 0x2c83, 0x1ce0, 0x0cc1,
    0xef1f, 0xff3e, 0xcf5d, 0xdf7c, 0xaf9b, 0xbfba, 0x8fd9, 0x9ff8,
    0x6e17, 0x7e36, 0x4e55, 0x5e74, 0x2e93, 0x3eb2, 0x0ed1, 0x1ef0
};

uint16_t crc16(const uint8_t *buf, size_t len) {
    uint16_t cksum = 0;
    for (size_t i = 0; i < len; i++) {
        cksum = crc16_tab[(((cksum >> 8) ^ buf[i]) & 0xFF)] ^ (cksum << 8);
    }
    return cksum;
}
```

Sending a Message with CRC

```
// 1. Encode protobuf message
uint8_t payload[64];
size_t protobuf_len = encode_protobuf_message(payload, sizeof(payload), &msg);

// 2. Calculate CRC over protobuf data
uint16_t crc = crc16(payload, protobuf_len);

// 3. Append CRC in big-endian order
payload[protobuf_len]     = (crc >> 8) & 0xFF;  // High byte
payload[protobuf_len + 1] = crc & 0xFF;         // Low byte

// 4. Total length includes CRC
size_t total_len = protobuf_len + 2;

// 5. Encode CAN identifier with total length
uint32_t identifier = encode_ext_id(sender, receiver, msg_id, total_len, true);

// 6. Send frame(s)
send_can_frame(identifier, payload, total_len);
```

Receiving a Message with CRC

```
// 1. Receive all frames and assemble payload
uint8_t payload[64];
size_t total_len = receive_message(payload);
```

```
// 2. Verify CRC (last 2 bytes)
size_t protobuf_len = total_len - 2;
uint16_t received_crc = (payload[protobuf_len] << 8) | payload[protobuf_len + 1];
uint16_t calculated_crc = crc16(payload, protobuf_len);

if (received_crc != calculated_crc) {
    // CRC mismatch - discard message
    return;
}

// 3. Decode protobuf (excluding CRC bytes)
decode_protobuf_message(payload, protobuf_len, &msg);
```

Identifier Encoding Example

To send a message from your device (ID=6) to VCU (ID=1) with Message ID 23 and payload length 4:

```
uint32_t identifier = (6 << 25)   // senderId = External Device
                    | (1 << 21)   // receiverId = VCU
                    | (23 << 13)  // messageId = GrabKeyResponseMsg
                    | (4 << 1)    // length = 4 bytes
                    | 1;          // isFirst = true
```

Identifier Decoding Example

```
uint32_t identifier = frame.identifier;
uint8_t senderId   = (identifier >> 25) & 0xF;
uint8_t receiverId = (identifier >> 21) & 0xF;
uint8_t messageId  = (identifier >> 13) & 0xFF;
uint16_t length    = (identifier >> 1) & 0xFFF;
bool isFirst       = identifier & 0x1;
```

Message Encoding (Protocol Buffers)

All messages are serialized using **Protocol Buffers (protobuf)** with the **nanopb** library, which is optimized for embedded systems.

Nanopb Library

- **Repository**: https://github.com/nanopb/nanopb
- **Documentation**: https://jpa.kapsi.fi/nanopb/docs/

Protobuf Definitions

```
syntax = "proto3";

enum State {
  OFF = 0;
  STAND_BY = 1;
  PARKING = 2;
  DRIVE = 3;
  SUPER = 4;
  CRUISE_CONTROL = 5;
  REVERSE = 6;
  LOCK = 7;
  TURTLE = 8;
}

enum GrabBikeModel {
  UNKNOWN = 0;
  QUANTUM_S2 = 1;
  QUANTUM_S3 = 2;
}

// Telemetry message - sent from VCU every 5 seconds
message GrabInfoMsg {
  float odo = 1;                                  // Mileage (ODO) in km
  float speed = 2;                                // Speed in km/h
  float soc = 3;                                  // State of Charge (0-100%)
  float cycleCount = 4;                           // Battery charge cycles (SOH
indicator)
  State state = 5;                                // Bike operating mode
  uint32 bikeError = 6;                           // BikeErrorCode flags (bitmask)
  uint32 protectionState = 7;                     // BMS protection state flags
  uint32 bmsSystemError = 8;                      // BMS system error flags
  bytes frameNumber = 9 [(nanopb).max_size = 20]; // Bike frame number in ASCII
(max 20 bytes)
  GrabBikeModel model = 10;                       // Bike model identifier
}

// Challenge-response protocol messages
message GrabKeyRequestMsg {
  // Empty message - VCU requests key
}

message GrabKeyResponseMsg {
  uint32 key = 1;                   // Shared secret key
}

message GrabChallengeMsg {
  uint32 token = 1;                 // Challenge token to hash
}

message GrabResponseMsg {
  uint32 hashedToken = 1;           // Hashed token = SHA256(key || token)[0:4]
}
```

```
// Deservice/reservice command messages
message GrabDeserviceCommandMsg {
  bool deservice = 1;              // true = deservice, false = reservice
}

message GrabDeserviceResponseMsg {
  bool success = 1;                // reservice: false if still deserviced by
failed challenges
  bool deserviced = 2;             // effective deservice state after handling
the command
}
```

API Messages

GrabInfoMsg (ID: 21)

Direction: VCU → External Device **Frequency**: Every 5 seconds **Purpose**: Transmit bike telemetry data

Fields

Field	Type	Description
odo	float	Total distance traveled (km)
speed	float	Current speed (km/h)
soc	float	Battery state of charge (0-100%)
cycleCount	float	Battery charge/discharge cycles (indicates battery health)
state	State enum	Current bike operating mode (see State enum above)
bikeError	uint32	Bike error flags (bitmask, see below)
protectionState	uint32	BMS protection flags (bitmask, see below)
bmsSystemError	uint32	BMS system error flags (bitmask, see below)
frameNumber	bytes	Bike frame number in ASCII (unique bike identifier)
model	GrabBikeModel	Bike model type

bikeError Flags

The `bikeError` field is a bitmask of error codes:

Bit	Error Code	Description
0	CONTROLLER_INCORRECT_VOLTAGE_READING	Controller voltage reading error
1	BMS_HIGH_DIFF_VOLTAGE	High voltage difference between battery cells

Bit	Error Code	Description
2	OTA_ROLLBACK_FAILED	Firmware update rollback failed
3	THROTTLE_BROKEN	Throttle sensor fault
4	LOW_MEMORY	System low memory
5	BMS_MODEL_MISMATCH	BMS model mismatch
6	VESC_MODEL_MISMATCH	Motor controller model mismatch
7	GRAB_DESERVICED	Bike is deserviced (failed authentication)

protectionState Flags (BMS Protection)

The `protectionState` field is a bitmask of BMS protection events:

Bit	Protection	Description
0	Cell_over_voltage	Individual cell over voltage
1	Cell_under_voltage	Individual cell under voltage
2	Pack_over_voltage	Total pack over voltage
3	Pack_under_voltage	Total pack under voltage
4	Pack_over_temperature	Battery over temperature
5	Pack_under_temperature	Battery under temperature
6	Mosfet_over_temperature	Power MOSFET over temperature
7	Balance_resistor_over_temperature	Balancing resistor over temperature
8	Charge_over_current	Charging current too high
9	Discharge_over_current	Discharge current too high
10	Hardware_cell_over_voltage	Hardware-level cell over voltage
11	Hardware_cell_under_voltage	Hardware-level cell under voltage
12	Hardware_discharge_over_current	Hardware-level discharge over current
13	Hardware_short_circuit	Short circuit detected
14	Hardware_alert_override	Hardware alert override
15	Hardware_device_x_ready	Hardware device not ready
16	User_charge_locked	Charging manually locked by user
17	User_discharge_locked	Discharging manually locked by user
18	Discharge_over_current_150A	Discharge over 150A
19	Discharge_over_current_180A	Discharge over 180A

Bit	Protection	Description
20	Discharge_over_current_210A	Discharge over 210A
21	Discharge_over_current_240A	Discharge over 240A
22	RegSource_under_voltage	Regulator source under voltage
23	Heartbeat_timeout	BMS heartbeat timeout

bmsSystemError Flags

The `bmsSystemError` field is a bitmask of BMS hardware errors:

Bit	Error	Description
0	Charge_FET_defective	Charge MOSFET is defective
1	Discharge_FET_defective	Discharge MOSFET is defective
2	Bottom_AFE_not_available	Bottom analog front-end chip unavailable
3	Top_AFE_not_available	Top analog front-end chip unavailable

---

Deservice/Reservice Command

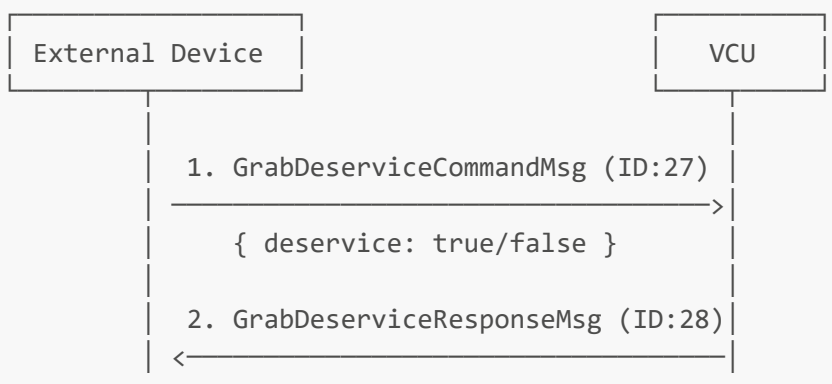
The device can remotely deservice (disable) or reservice (re-enable) the bike with an explicit command. A deserviced bike is forced to PARKING at standstill and cannot enter a driving mode — the same enforcement as a challenge-driven deservice.

The bike is deserviced whenever **either** source is active:

- **Failed challenges** — the device stops answering, so the bike deservices after 5 consecutive failures (recovered by a correct challenge response).
- **Deservice command** — an explicit command from the device (recovered only by a reservice command).

A reservice command clears only the command source. If the bike is also deserviced by failed challenges, it stays deserviced until the device answers a challenge correctly.

Protocol Flow



```
          |      { success, deserviced }        |
          |                                     |
```

The response is sent immediately (typically within 100 ms). The effective deservice state is also visible in every subsequent `GrabInfoMsg` (`bikeError` bit 7, `GRAB_DESERVICED`), so it can be confirmed continuously through telemetry.

Behavior

Command	Result
Deservice	Bike is deserviced, `success = true`, `deserviced = true`
Reservice, bike deserviced by command only	Bike is serviced, `success = true`, `deserviced = false`
Reservice, bike still deserviced by failed challenges	Command source cleared, `success = false`, `deserviced = true`

Additional notes:

- Commands are idempotent — repeating a command that is already satisfied returns the same result.
- Command deservice persists across power cycles: a bike deserviced by command stays deserviced after the battery is disconnected and reconnected, until a reservice command (or toggling Grab mode).
- Enforcement happens at standstill — a deservice command sent while the bike is moving takes effect once the bike stops, via the same mechanism as challenge deservice. No motion check is needed on the command.
- If no response is received (e.g. a lost CAN frame), resend the command — retrying is always safe.
- The command is only handled when the bike has completed the key exchange with the device.

Deservice/Reservice Protocol

The deservice/reservice protocol uses a secure challenge-response mechanism to verify the external device is authorized. If the device fails to respond correctly **5 consecutive times**, the bike enters a "deserviced" state and cannot be driven.

Protocol Flow



```
      |  [Device stores the key]                  |
      |                                           |
      |  3. GrabChallengeMsg (ID: 24)             |
      | ------------------------------------------>|  (every 120s)
      |       { token: <uint32> }                 |
      |                                           |
      |  4. GrabResponseMsg (ID: 25)              |
      | <------------------------------------------|
      |       { hashedToken: SHA256(key||token) } |
      |                                           |
      |  [If valid: stay serviced]                |
      |  [If 5 consecutive failures: DESERVICED]  |
      |                                           |
```

Hashing Algorithm

The response uses **SHA-256** hashing with the stored key and the token from the challenge message:

```
hashedToken = first_4_bytes( SHA256( key || token ) )
```

Where:

- `key`: 32-bit unsigned integer from key exchange (stored by device)
- `token`: 32-bit unsigned integer received in `GrabChallengeMsg`
- `||`: Concatenation (big-endian byte order)

Implementation Example (C)

```c
#include <stdint.h>
#include <string.h>

// Use any SHA-256 library (e.g., mbedtls, OpenSSL, etc.)
// This example assumes a sha256() function exists

uint32_t computeHashedToken(uint32_t key, uint32_t token) {
    uint8_t input[8];

    // Key bytes (big-endian)
    input[0] = (key >> 24) & 0xFF;
    input[1] = (key >> 16) & 0xFF;
    input[2] = (key >> 8) & 0xFF;
    input[3] = key & 0xFF;

    // Token bytes (big-endian)
    input[4] = (token >> 24) & 0xFF;
    input[5] = (token >> 16) & 0xFF;
    input[6] = (token >> 8) & 0xFF;
    input[7] = token & 0xFF;
```

```
    uint8_t hash[32];
    sha256(input, 8, hash);  // Your SHA-256 implementation

    // Return first 4 bytes as big-endian uint32
    return ((uint32_t)hash[0] << 24) |
           ((uint32_t)hash[1] << 16) |
           ((uint32_t)hash[2] << 8)  |
           (uint32_t)hash[3];
}
```

Timing Requirements

Event	Timeout
Key request retry (until a response arrives)	5 seconds
Challenge response	5 seconds
Challenge interval	120 seconds
Consecutive failures for deservice	5

Deservice Behavior

When the bike enters DESERVICED state (bit 7 in `bikeError` field):

- Bike is forced to PARKING mode
- Cannot enter DRIVE, SUPER, REVERSE, or CRUISE modes
- Error is displayed on LCD
- Bike continues sending challenges every 120 seconds
- Correct response clears DESERVICED and restores normal operation

---

Test Board Usage

A pre-programmed test board is provided that emulates a bike's VCU for development and testing. This allows you to test your device implementation without a real bike.

Connecting to the Test Board

1. Connect your device to the test board's CAN connector
2. Power on the test board
3. Connect a micro USB cable to view serial output for debugging (115200 baud)

> **Note on CAN Termination**: Unlike the bike (which has both 120Ω termination resistors), the test board only has **one 120Ω resistor**. You must add a 120Ω termination resistor on your device when connecting to the test board.

Test Board Behavior

The test board:

1. **Broadcasts mock telemetry** (`GrabInfoMsg`) every 5 seconds
2. **Requests a key** from your device on startup (if no key is stored)
3. **Sends challenges and verifies responses** every 2 seconds
4. **Shows the expected hash** in serial output for debugging
5. **Requires 5 consecutive failures** before entering DESERVICED state
6. **Handles deservice/reservice commands** (`GrabDeserviceCommandMsg`), with the command deservice persisted across power cycles, like a real bike

> **Note**: The test board uses a shorter challenge interval (2s) compared to the real bike (120s) for faster development iteration.

Serial Output Example

Protocol events are printed as ESP-IDF log lines; the `[timestamp][level][file:line] function():` prefix is abbreviated to `[level]` below.

```
========================================
  GrabCanTestBoard - Bike Emulator
========================================
Press BOOT button (GPIO0) to reset key

No key in NVS. Will request from device.
Initializing CAN bus...
CAN bus initialized
Setup complete. Tasks started.

[I] Requesting key from Grab device
[I] Grab key received and stored: 2847593821

[INFO] ODO=1234.5 SPD=0.0 SOC=85.0 State=2 BikeError=0
[INFO] ODO=1234.5 SPD=0.0 SOC=85.0 State=2 BikeError=0
...
[I] Sending challenge, token: 0, expecting hash: 1938472650
[I] Grab challenge verified, token: 1
...
[I] Grab deservice command: success 1, deserviced 1
[INFO] ODO=1234.5 SPD=0.0 SOC=85.0 State=2 BikeError=128
[I] Grab reservice command: success 1, deserviced 0
...
[I] Sending challenge, token: 1, expecting hash: 3847261940
[W] Grab response mismatch! Expected: 3847261940, Got: 12345 (failure 1/5)
...
[W] Grab response mismatch! Expected: 1284736251, Got: 12345 (failure 5/5)
[INFO] ODO=1234.5 SPD=0.0 SOC=85.0 State=2 BikeError=128
```

A challenge that gets no answer within 5 seconds logs `No response from Grab device (failure n/5)` instead of the mismatch line. Deservice shows as bit 7 (value 128) in the `[INFO]` line's `BikeError`.

Resetting the Key

Press the **BOOT button** on the test board to clear the stored key and request a new one from your device. This is useful for testing the key exchange flow.

Test Scenarios

1. **Normal Operation**
    - Connect your device to the test board
    - Wait for key request, respond with a random key
    - Store the key
    - Respond to challenge messages with correct hashed tokens
    - Verify `bikeError` field remains 0
2. **Deservice Test**
    - Respond with an incorrect hash or don't respond within 5 seconds
    - After 5 consecutive failures, verify `bikeError` field shows bit 7 set (value 128 or higher)
    - The serial output will show "(failure 5/5)"
3. **Reservice Test**
    - After being deserviced, respond with the correct hashed token
    - Verify `bikeError` bit 7 is cleared
    - The serial output will show "Grab challenge verified"
4. **Key Reset Test**
    - Press BOOT button on test board
    - Complete new key exchange
    - Verification continues with new key
5. **Deservice/Reservice Test**
    - Send `GrabDeserviceCommandMsg` with `deservice = true`
    - Verify `GrabDeserviceResponseMsg` shows `success = true` and `deserviced = true`
    - Verify subsequent `GrabInfoMsg` shows `bikeError` bit 7 set (value 128)
    - Power-cycle the test board and verify it comes back deserviced (bit 7 still set)
    - Send `GrabDeserviceCommandMsg` with `deservice = false`
    - Verify the response shows `success = true` and `deserviced = false`

Support

For technical questions or integration support, please contact your Dat Bike technical representative.
